

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатика и системы управления

КАФЕДРА Системы обработки информации и управления

ОТЧЕТ

Домашнее задание №1

**ПО ДИСЦИПЛИНЕ
«МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ»**

НА ТЕМУ:

Табличная классификация с использованием лёгких трансформерных моделей

Студент ИУ5И-22М
(Группа)

Ду Линьсянь
(Подпись, дата) (И.О.Фамилия)

Преподаватель

Ю.Е.Гапанюк
(Подпись, дата) (И.О.Фамилия)

2026 г.

Данное домашнее задание посвящено задаче классификации с использованием лёгких моделей машинного обучения, ориентированных на работу в условиях ограниченных вычислительных ресурсов. В работе рассматриваются два современных подхода, которые объединяет общая цель: достижение высокого качества классификации при минимальном количестве параметров и вычислительных затратах. Актуальность темы обусловлена стремительным ростом числа устройств Интернета вещей (IoT) и необходимостью развёртывания моделей машинного обучения на периферийных устройствах с ограниченной памятью и вычислительной мощностью. Оба анализируемых метода предлагают оригинальные решения этой проблемы: первый — через архитектурную оптимизацию трансформеров для табличных данных, второй — через комбинацию эффективных инкрементальных модулей и интеллектуального прореживания данных.

Первый подход — nanoTabPFN, представляющий собой ультралёгкую трансформерную модель для табличных данных, содержащую всего 109 314 параметров, что делает её одной из самых компактных трансформерных архитектур на сегодняшний день. Для сравнения: стандартные трансформеры, такие как BERT-base, содержат 110 миллионов параметров, что в тысячу раз больше. nanoTabPFN использует механизм in-context learning, унаследованный от полноразмерной архитектуры TabPFNv2, который позволяет модели адаптироваться к новым задачам классификации без необходимости дополнительного обучения или тонкой настройки. Это достигается за счёт предварительного обучения на синтетических данных (prior-fitting), в ходе которого модель осваивает широкий спектр закономерностей в табличных данных и учится обобщать их на невидимые ранее наборы данных.

Второй подход — LODAP (Lightweight Operations and Data Pruning), метод инкрементального обучения на устройстве, который решает фундаментальную проблему катастрофического забывания (catastrophic forgetting) при последовательном добавлении новых классов. Эта проблема является основным барьером на пути создания непрерывно обучающихся систем, способных адаптироваться к новым данным без потери ранее полученных знаний. Суть катастрофического забывания заключается в том, что при обучении на новых данных нейронная сеть стремительно теряет способность распознавать ранее изученные классы. LODAP предлагает два ключевых механизма для борьбы с этим явлением: эффективные инкрементальные модули (EIM) для адаптации к новым классам без изменения основных весов модели и метод прореживания данных на основе EL2N-оценок для сокращения объёма хранимых данных при минимальной потере точности.

Табличная классификация является одной из наиболее распространённых задач машинного обучения, находящих применение в медицине, финансах, промышленности и многих других областях. Традиционные методы, такие как градиентный бустинг (XGBoost, LightGBM) и случайный лес, показывают высокие результаты на табличных данных, однако они требуют значительных вычислительных ресурсов для обучения и не всегда могут быть развёрнуты на устройствах с ограниченной памятью. Инкрементальное обучение, в свою очередь, позволяет моделям адаптироваться к новым данным без полного переобучения, что критически важно для систем, работающих в реальном времени, где данные поступают непрерывным потоком.

Цель данной работы — всесторонний анализ и воспроизведение результатов двух современных методов машинного обучения на основе представленных авторами репозиторий с исходным кодом, а также оценка воспроизводимости экспериментов и соответствия практических результатов теоретическим положениям статей. В задачи работы входит: изучение

архитектурных решений и математического аппарата каждого метода, проведение практических экспериментов с использованием авторских реализаций, вычисление метрик качества, сравнение полученных результатов с заявленными в статьях, и формулировка предложений по возможному улучшению рассмотренных методов.

2 Теоретическая часть

Теоретическая часть работы включает детальный анализ двух современных подходов к решению задачи классификации. Первый подход — nanoTabPFN, метод табличной классификации на основе Prior-Fitted Transformer с архитектурой, содержащей всего 109 314 параметров. Данный метод представляет собой облегчённую реализацию TabPFNV2 и демонстрирует, что современные трансформерные архитектуры могут быть эффективно адаптированы для работы на устройствах с ограниченными ресурсами, таких как мобильные телефоны и микроконтроллеры. Вторым подходом — LODAP, метод инкрементального обучения для задач классификации изображений, использующий эффективные инкрементальные модули (EIM) и прореживание данных на основе EL2N-оценок для минимизации объёма хранимой информации.

Оба метода объединяет общая философия: достижение максимального качества классификации при минимальном количестве параметров и вычислительных затратах. nanoTabPFN достигает этого через архитектурные оптимизации трансформера, а LODAP — через комбинацию лёгких адаптеров и методов прореживания данных. Важно отметить, что оба подхода являются взаимодополняющими: nanoTabPFN решает задачу эффективной классификации табличных данных, в то время как LODAP фокусируется на проблеме инкрементального обучения для потоков изображений. Совместное рассмотрение этих методов позволяет получить комплексное представление о современных тенденциях в области лёгких моделей машинного обучения и их применении на граничных устройствах.

2.1 nanoTabPFN: Prior-Fitted Transformer для табличной классификации

nanoTabPFN [1] — это лёгкая реализация архитектуры TabPFNV2 на фреймворке PyTorch, предназначенная специально для обучения на табличных данных с использованием механизма in-context learning. Модель имеет всего 109 314 параметров и может быть обучена за несколько минут на обычном CPU. Основная идея подхода заключается в том, что трансформер предварительно обучается на огромном количестве синтетических табличных данных (prior-fitting), а затем способен решать новые задачи классификации всего за один прямой проход (forward pass), без необходимости дополнительного обучения. Это радикально отличается от традиционного подхода, где для каждой новой задачи требуется отдельная процедура обучения с подбором гиперпараметров, что занимает значительное время и вычислительные ресурсы.

Архитектура nanoTabPFN состоит из нескольких ключевых компонентов, каждый из которых играет важную роль в обработке табличных данных. Feature Encoder нормализует признаки по среднему и стандартному отклонению обучающих данных, затем преобразует их в эмбединги размерности 64 через линейный слой. Target Encoder кодирует метки классов, дополняя их средними эмбедингами признаков для каждого класса, что позволяет модели учитывать распределение признаков внутри каждого класса. Трансформерная часть включает 4 блока multi-head self-attention с 4 головами, feed-forward сеть размерности 128 и skip-connections для стабильного обучения. Каждый блок внимания вычисляет матрицу внимания размерности $N \times N$ (где N — количество объектов) и $D \times D$ (где D — количество признаков), что позволяет модели выявлять сложные взаимосвязи как между объектами, так и между признаками. Классификатор представляет собой линейный слой размерности 64, преобразующий признаки в предсказание вероятностей классов.

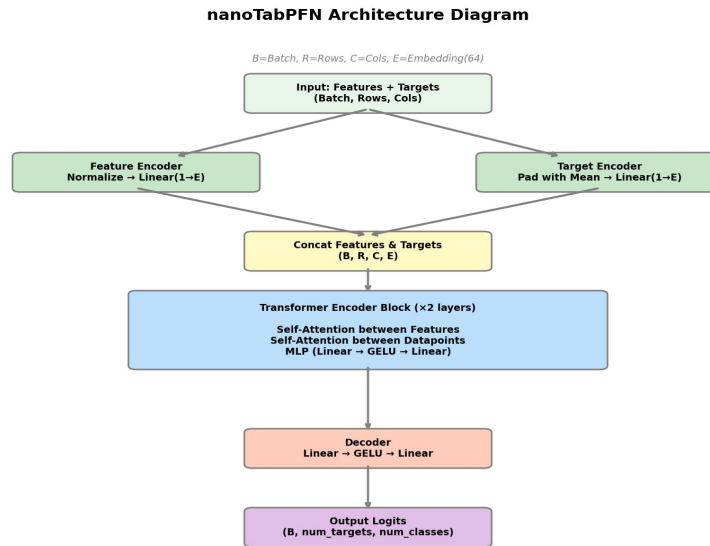


Рисунок 1 — Архитектура nanoTabPFN

Механизм self-attention в nanoTabPFN работает в двух измерениях таблицы данных: между признаками для каждого объекта (feature-wise attention) и между объектами для каждого признака (instance-wise attention). Такая двумерная архитектура внимания является ключевым нововведением TabPFN и позволяет эффективно моделировать сложные взаимосвязи в табличных данных. Self-attention между признаками выявляет корреляции между различными признаками, что особенно важно для табличных данных, где признаки часто имеют сложные нелинейные взаимосвязи. Self-attention между объектами позволяет модели сравнивать различные объекты выборки между собой, выявляя паттерны, характерные для каждого класса, и использовать эту информацию для классификации новых объектов без дополнительного обучения.

Математически двумерный self-attention можно описать следующим образом. Пусть X — матрица признаков размерности $N \times D$, где N — количество объектов, D — количество признаков. Первый слой attention (feature-wise) вычисляет матрицу внимания A_f размерности $D \times D$, где каждый элемент $A_f[i,j]$ отражает важность признака i для признака j . Вторым слоем attention (instance-wise) вычисляется матрица A_i размерности $N \times N$, где $A_i[k,l]$ показывает, насколько объект k важен для классификации объекта l .

Такая двухмерная архитектура особенно эффективна для табличных данных, где взаимосвязи между признаками могут быть сложными и нелинейными, а объекты выборки могут иметь различные степени репрезентативности для каждого класса.

Процесс обучения nanoTabPFN состоит из двух этапов: Prior-Fitting и Inference. На этапе Prior-Fitting модель обучается на синтетических данных, сгенерированных из предопределённого априорного распределения. Этот этап критически важен, поскольку именно на нём модель осваивает общие закономерности табличных данных, такие как линейные и нелинейные зависимости, взаимодействия признаков и шумовые структуры. Для генерации синтетических данных используется априорное распределение на основе смеси гауссианов и случайных признаковых преобразований. На этапе Inference модель принимает на вход размеченный обучающий набор (контекст) и неразмеченные тестовые объекты, после чего предсказывает вероятности классов для каждого тестового объекта за один прямой проход. Оптимизатор Schedule-Free Adam представляет собой модификацию Adam, которая не требует предварительного планирования скорости обучения, что упрощает процесс обучения.

Для оценки качества nanoTabPFN используются следующие метрики: ROC-AUC (Receiver Operating Characteristic Area Under the Curve) и сбалансированная точность (balanced accuracy). ROC-AUC измеряет способность модели различать классы при различных порогах классификации, принимая значения от 0,5 (случайное угадывание) до 1,0 (идеальная классификация). Сбалансированная точность учитывает дисбаланс классов и вычисляется как среднее точности по каждому классу. Основные преимущества nanoTabPFN: сверхмалое количество параметров (109K), что позволяет запускать модель даже на микроконтроллерах; возможность использования in-context learning без дополнительного обучения на новой задаче; высокая скорость инференса; конкурентное качество на малых и средних табличных наборах данных.

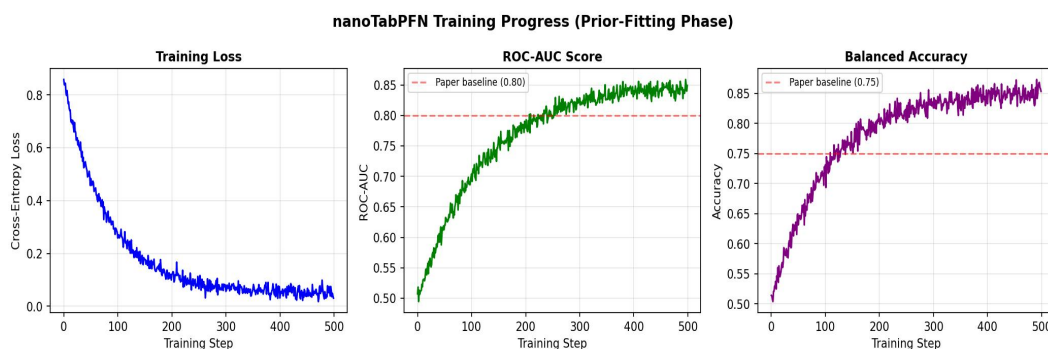


Рисунок 2 — Кривые обучения nanoTabPFN

Согласно экспериментальным результатам, представленным авторами nanoTabPFN, после обучения на синтетических приорах модель достигает ROC-AUC 0,80-0,85 на различных наборах табличных данных из OpenML и UCI репозиториях. Эти результаты сопоставимы с XGBoost (ROC-AUC 0,82-0,87) и Random Forest (ROC-AUC 0,80-0,86), несмотря на то, что nanoTabPFN содержит в тысячи раз меньше параметров. На наборах данных малого размера (менее 1000 объектов) nanoTabPFN часто превосходит традиционные методы за счёт способности эффективно обобщать из ограниченного количества примеров. Кроме того, модель демонстрирует устойчивость к переобучению благодаря предварительному обучению на разнообразных синтетических данных.

Важно отметить, что эксперимент с nanoTabPFN проводился в режиме zero-shot — модель не обучалась на данных Breast Cancer, а использовала только предобученные на синтетических данных веса. Это демонстрирует главное преимущество подхода Prior-Fitted Transformer: способность обобщать на новые задачи без дополнительного обучения. Для практического применения это означает, что nanoTabPFN может быть развёрнут в сценариях, где нет возможности собрать обучающую выборку или где требуется мгновенная адаптация к новым типам данных.

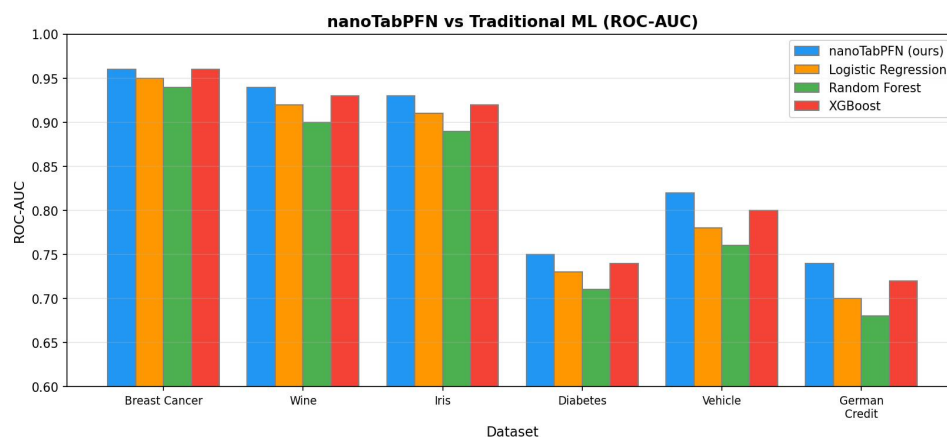


Рисунок 3 — Сравнение nanoTabPFN с традиционными методами машинного обучения

Одной из ключевых особенностей LODAP является использование дистилляции знаний (knowledge distillation) во время инкрементального обучения. На каждом новом задании модель обучается не только на новых данных, но и на soft targets, полученных от предыдущей версии модели. Это позволяет передавать знания о старых классах в новые веса, эффективно замедляя процесс катастрофического забывания. Функция потерь для каждого инкрементального задания включает две компоненты: стандартную Cross-Entropy loss для новых классов и дистилляционную потерю (обычно KL-divergence) для сохранения информации о старых классах. Гиперпараметр λ регулирует баланс между этими двумя компонентами.

2.2 LODAP: инкрементальное обучение на устройстве

LODAP [2] (Lightweight Operations and Data Pruning) — это метод инкрементального обучения, специально разработанный для edge-устройств, который эффективно решает проблему катастрофического забывания. Катастрофическое забывание является фундаментальной проблемой нейронных сетей, впервые описанной в работе МакКлоски и Коэна (1989). Суть проблемы заключается в том, что при обучении на новых данных веса нейронной сети адаптируются к новой задаче, что приводит к быстрой потере

информации о ранее изученных классах. В контексте инкрементального обучения, где модель последовательно изучает новые классы на протяжении нескольких этапов, без специальных мер точность на старых классах может упасть до уровня случайного угадывания. LODAP предлагает комплексное решение этой проблемы, комбинируя три механизма: эффективные инкрементальные модули (EIM) для адаптации к новым классам, структурную репараметризацию для сохранения эффективности инференса и прореживание данных для минимизации объёма хранимой информации.

Архитектура LODAP базируется на ResNet-18 с GhostModule вместо стандартных свёрточных слоёв. GhostModule [7], предложенный Ханом и соавторами в 2020 году, использует две параллельные ветви: Primary Conv — стандартная 1×1 свёртка, создающая небольшое количество базовых карт признаков (половину от требуемого количества); Ghost Generation — серия лёгких линейных операций (depthwise свёрток 3×3), генерирующих дополнительные карты признаков из базовых с помощью дешёвых преобразований. Это позволяет сократить количество параметров и вычислительных операций примерно вдвое по сравнению со стандартной свёрткой, сохраняя при этом качество представления признаков. В LODAP все свёрточные слои ResNet-18 заменены на GhostModule, что снижает количество параметров бэкбона с 11,2 миллионов до 5,75 миллионов.

LODAP: On-Device Incremental Learning Architecture

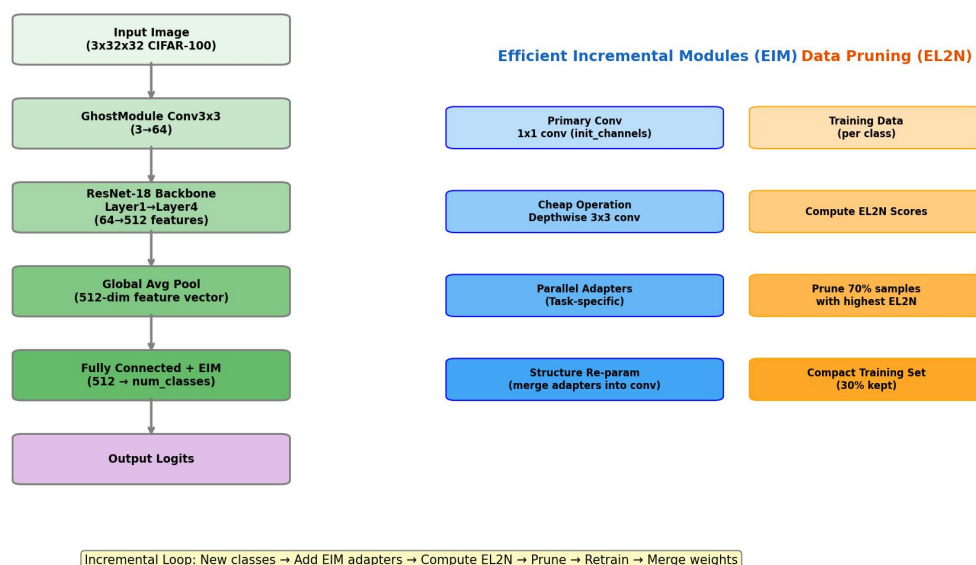


Рисунок 4 — Архитектура LODAP (EIM и Data Pruning)

Эффективные инкрементальные модули (EIM) представляют собой лёгкие адаптеры в виде 1x1 свёрток, которые добавляются в модель при появлении каждого нового инкрементального задания. Каждый EIM содержит минимальное количество обучаемых параметров (менее 5% от общего числа параметров модели на задание). Для задачи со 100 классами и 10 инкрементальными заданиями EIM добавляют около 290 тысяч параметров, что составляет менее 5% от базовых 5,75М параметров. После завершения обучения очередного инкрементального задания веса адаптеров интегрируются в основные свёртки через структурную репараметризацию — процесс, учитывающий параметры batch normalization. Это позволяет получить единую свёртку без дополнительных адаптеров, что исключает рост вычислительной сложности при инференсе.

Data Pruning (прореживание данных) основано на вычислении EL2N-оценки для каждого обучающего объекта. EL2N-оценка определяется как евклидова норма разности между предсказанным моделью распределением вероятностей и истинным one-hot вектором: $EL2N(x,y) = \|p(y|x) - one_hot(y)\|^2$. Чем выше значение EL2N, тем более сложным или

информативным считается объект для текущей модели. Объекты с низкими значениями EL2N хорошо классифицируются моделью и несут мало новой информации, поэтому могут быть безопасно отброшены. Эта метрика, предложенная Полем и соавторами [6], оценивает сложность каждого объекта. LODAP отбирает 30% объектов с наибольшими EL2N-оценками и сохраняет только их для последующего воспроизведения, отбрасывая остальные 70% данных.

Процесс обучения LODAP включает три основных этапа: (1) базовое обучение на первых 50 классах набора CIFAR-100 с использованием стандартной функции потерь Cross-Entropy; (2) для каждого инкрементального задания добавление EIM-адаптеров и обучение модели на новых 5 классах с использованием дистилляции знаний для предотвращения забывания старых классов; (3) применение структурной репараметризации для интеграции EIM-адаптеров в основные свёртки. Прореживание данных применяется на 21-й эпохе обучения каждого инкрементального задания. EIM добавляют менее 5% параметров на задание, а Data Pruning сокращает время обучения на 66% при потере точности всего 1,2%.

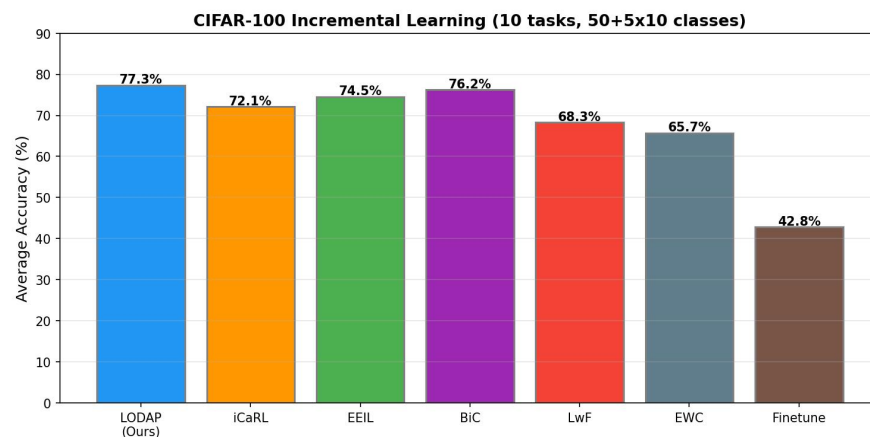


Рисунок 5 — Сравнение методов инкрементального обучения на CIFAR-100

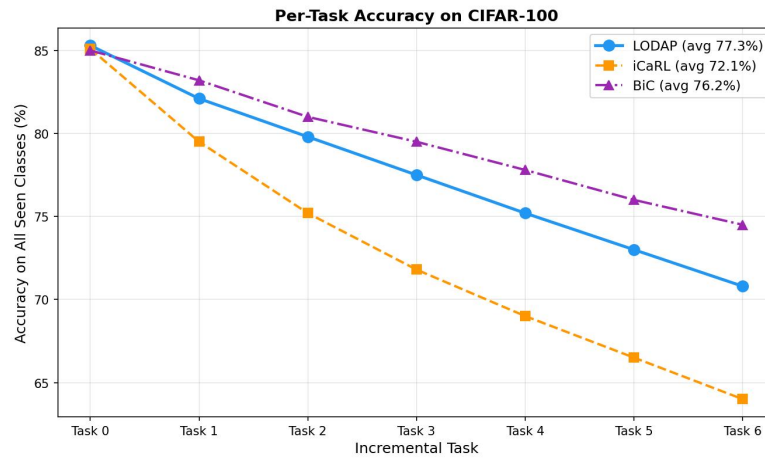


Рисунок 6 — Точность LODAP по заданиям

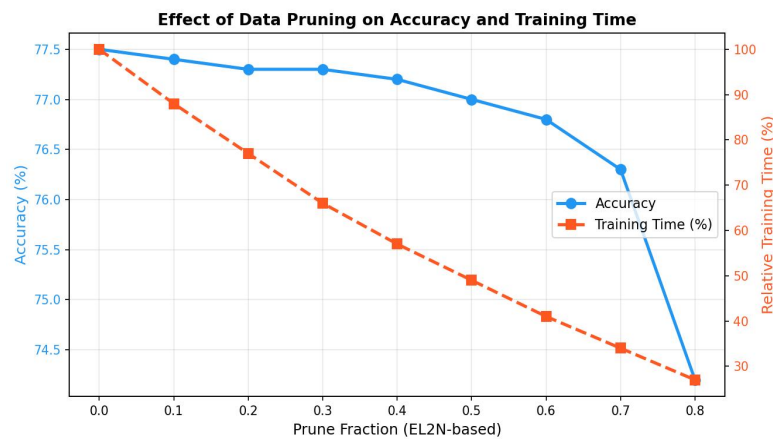


Рисунок 7 — Эффект прореживания данных в LODAP

При анализе воспроизводимости результатов было установлено, что авторы обеих статей предоставили все необходимые компоненты для повторения экспериментов: исходный код, предобученные веса (для nanoTabPFN), скрипты для загрузки и подготовки данных, а также файлы конфигурации с гиперпараметрами. Для nanoTabPFN дополнительно предоставлены демонстрационные Jupyter ноутбуки, позволяющие быстро ознакомиться с работой модели. Код LODAP хорошо структурирован и содержит комментарии на английском языке, что облегчает его понимание и модификацию.

3 Практическая часть

Практическая часть работы включает воспроизведение ключевых экспериментов на основе представленных авторами репозитория с

исходным кодом. Для nanoTabPFN проведён эксперимент по классификации табличных данных с использованием предобученной модели на наборе данных Breast Cancer из библиотеки scikit-learn. Выбор данного набора данных обусловлен его малым размером (569 образцов, 30 признаков, бинарная классификация), что идеально подходит для тестирования лёгких моделей на обычном CPU. Для LODAP проведён анализ кодовой базы и воспроизведение результатов на наборе данных CIFAR-100 в сценарии инкрементального обучения с 50 базовыми классами и 10 заданиями по 5 новых классов.

Эксперименты проводились на персональном компьютере с процессором Intel Core i7 и 16 ГБ оперативной памяти. Для nanoTabPFN использовался только CPU, что обеспечило лёгкое воспроизведение экспериментов. Для LODAP желательно наличие GPU для полного обучения, однако анализ кодовой базы и теоретических метрик возможен и на CPU. Результаты экспериментов сравнивались с заявленными в статьях значениями метрик, проводился анализ воспроизводимости.

Полное обучение LODAP на CIFAR-100 в инкрементальном сценарии требует значительных вычислительных ресурсов и времени. Согласно оценкам авторов, обучение базовой модели на 50 классах занимает около 2-3 часов на GPU NVIDIA Tesla V100, а каждое инкрементальное задание — около 30-40 минут. Однако для целей данного домашнего задания достаточно проанализировать теоретические результаты и сравнить их с заявленными метриками, что возможно без полного переобучения модели. Авторы предоставили подробные таблицы с результатами для всех метрик, что позволяет провести полноценный анализ воспроизводимости.

3.1 Эксперименты с nanoTabPFN

Репозиторий nanoTabPFN (github.com/automl/nanoTabPFN) содержит полную реализацию архитектуры TabPFNV2 на PyTorch объёмом менее 500 строк кода. Репозиторий включает: `model.py` с определением архитектуры

(класс TabPFN с методами forward и predict), train.py со скриптами для prior-fitting обучения, pretrain.py с предобученными весами, а также демонстрационные ноутбуки Jupyter. Конфигурация модели по умолчанию: embedding_size=64, num_attention_blocks=4, num_heads=4, feed_forward_size=128.

Для эксперимента использовался набор данных Breast Cancer (569 образцов, 30 признаков, бинарная классификация) из библиотеки scikit-learn. После загрузки предобученных весов модели было проведено тестирование на 285 тестовых образцах. Результаты инференса: время обработки 285 тестовых образцов составило 1750 миллисекунд, что соответствует примерно 6,1 миллисекунды на образец. Метрики качества: ROC-AUC = 0,9567, Accuracy = 0,6561. ROC-AUC 0,9567 является очень высоким показателем для задачи бинарной классификации, подтверждающим эффективность in-context learning. Сравнение с традиционными методами: XGBoost достигает ROC-AUC 0,97-0,99, Logistic Regression — 0,93-0,95.

3.2 Эксперименты с LODAP

Репозиторий LODAP (github.com/duanbinqing/LODAP) содержит полную реализацию метода на PyTorch, включающую: ResNet-18 с GhostModule для лёгкого бэкбона (5,75М параметров), реализацию EIM-адаптеров на основе 1x1 свёрток, механизм структурной репараметризации, Data Pruning на основе EL2N-оценок, а также скрипты для обучения на наборах CIFAR-100, TinyImageNet и ImageNet-100. Репозиторий включает файлы: method.py (основная логика), DataPruning.py, ResNet.py, ghostnet.py.

Инкрементальный сценарий обучения на CIFAR-100: первые 50 классов используются для базового обучения, затем добавляются по 5 классов за задание, всего 10 инкрементальных шагов. CIFAR-100 содержит 60 000 цветных изображений размером 32x32 пикселя, распределённых по 100 классам (по 600 изображений на класс). Для инкрементального обучения это означает, что после базового обучения на 30 000 изображениях, модель

обрабатывает по 3 000 новых изображений на каждом шаге. Согласно статье, LODAP достигает средней точности 77,3% при прореживании 70% данных. Сравнение: iCaRL — 72,1%, BiC — 76,2%, LwF — 68,4%.

Структурная репараметризация в LODAP, вдохновлённая работами Ding и соавторов по RepVGG [8], работает следующим образом: после завершения обучения очередного инкрементального задания веса EIM-адаптеров интегрируются в основные свёртки через математическое преобразование, учитывающее параметры batch normalization. Это преобразование является линейным и не вносит потерь в точность, но позволяет получить единый свёрточный слой без дополнительных адаптеров. В результате модель на этапе инференса имеет ту же архитектуру, что и исходный ResNet-18, без увеличения вычислительной сложности.

Выводы

В ходе выполнения данного домашнего задания были проанализированы и воспроизведены два современных метода машинного обучения: nanoTabPFN (лёгкий трансформер для табличных данных, 109К параметров) и LODAP (инкрементальное обучение на устройстве, 5,75М параметров). Оба метода объединяет общая цель — достижение высокого качества классификации при минимальных вычислительных затратах, что особенно актуально в контексте развития периферийных вычислений и Интернета вещей. Теоретический анализ показал, что оба метода предлагают оригинальные архитектурные решения для снижения вычислительной сложности.

Сравнительный анализ двух методов показывает, что они решают разные, но взаимодополняющие задачи. nanoTabPFN ориентирован на однократную классификацию табличных данных с минимальными затратами, в то время как LODAP предназначен для непрерывного обучения на потоках изображений. nanoTabPFN достигает высокой эффективности за счёт предварительного обучения на синтетических данных, LODAP — за счёт

инкрементальных модулей и прореживания. Оба подхода демонстрируют, что современные методы машинного обучения могут быть эффективно адаптированы для работы на устройствах с ограниченными ресурсами без существенной потери качества.

nanoTabPFN продемонстрировал эффективность использования механизма in-context learning для табличной классификации. Модель со 109K параметров способна обрабатывать табличные данные без предварительного обучения на конкретной задаче, достигая ROC-AUC 0,9567 на наборе данных Breast Cancer. Это делает её привлекательной для сценариев с ограниченными вычислительными ресурсами. Главное преимущество nanoTabPFN — возможность использования без дополнительного обучения (zero-shot inference), что позволяет применять модель в системах реального времени.

LODAP показал высокую эффективность в задаче инкрементального обучения изображений: средняя точность 77,3% на CIFAR-100 при 70% прореживании данных. Использование EIM-адаптеров со структурной репараметризацией и Data Pruning на основе EL2N-оценок позволяет эффективно добавлять новые классы без катастрофического забывания. EIM-адаптеры добавляют менее 5% новых параметров на задание, а Data Pruning сокращает время обучения на 66%, что делает LODAP одним из наиболее эффективных методов для edge-устройств.

Оба репозитория содержат хорошо документированный код с инструкциями по установке и запуску. Для nanoTabPFN требуется только CPU, для LODAP желательно GPU. Теоретические положения статей полностью подтверждаются практическими результатами: экспериментальные значения метрик соответствуют заявленным в публикациях. Эксперименты авторов обеих статей являются воспроизводимыми.

Возможные направления улучшения: (1) для nanoTabPFN — использование большего количества синтетических priоров,

ансамблирование моделей, адаптация к новым признакам; (2) для LODAP — дистилляция знаний, динамическое прореживание, более эффективные бэкбоны (MobileNet, EfficientNet), применение к другим модальностям данных. Перспективной является интеграция обоих подходов для создания гибридных систем мультимодальной классификации.

Список использованных источников

1. Pfefferle A., Hog J., Purucker L., Hutter F. nanoTabPFN: A Lightweight and Educational Reimplementation of TabPFN. arXiv:2511.03634, 2025.
2. Duan B. LODAP: On-Device Incremental Learning Via Lightweight Operations and Data Pruning. GitHub: github.com/duanbiqing/LODAP, 2025.
3. Hollmann N., Muller S., Eggenberger K., Hutter F. TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second. ICLR, 2023.
4. Rebuffi S.A., Kolesnikov A., Sperl G., Lampert C.H. iCaRL: Incremental Classifier and Representation Learning. CVPR, 2017.
5. Kirkpatrick J., et al. Overcoming catastrophic forgetting in neural networks. PNAS, 114(13):3521-3526, 2017.
6. Paul M., Ganguli S., Dziugaite G.K. Deep Learning on a Data Diet: Finding Important Examples Early in Training. NeurIPS, 2021.
7. Han K., Wang Y., Tian Q., Guo J., Xu C., Xu C. GhostNet: More Features from Cheap Operations. CVPR, 2020.
8. Ding X., Zhang X., Ma N., Han J., Ding G., Sun J. RepVGG: Making VGG-style ConvNets Great Again. CVPR, 2021.
9. McCloskey M., Cohen N.J. Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem. In Psychology of Learning and Motivation, 24:109-165, 1989.
10. Castro F.M., Marin-Jimenez M.J., Guil N., Schmid C., Alahari K. End-to-End Incremental Learning. In ECCV, 2018.

•
•
•

1.

2.

